Name : Vansh Sharma Roll no : 2401109 Subject : ASDD Class : T22

Lab outcomes :
LO4: Containerize, orchestrate, and deploy applications using Docker, Docker Compose, Kubernetes, and cloud infrastructure tools.

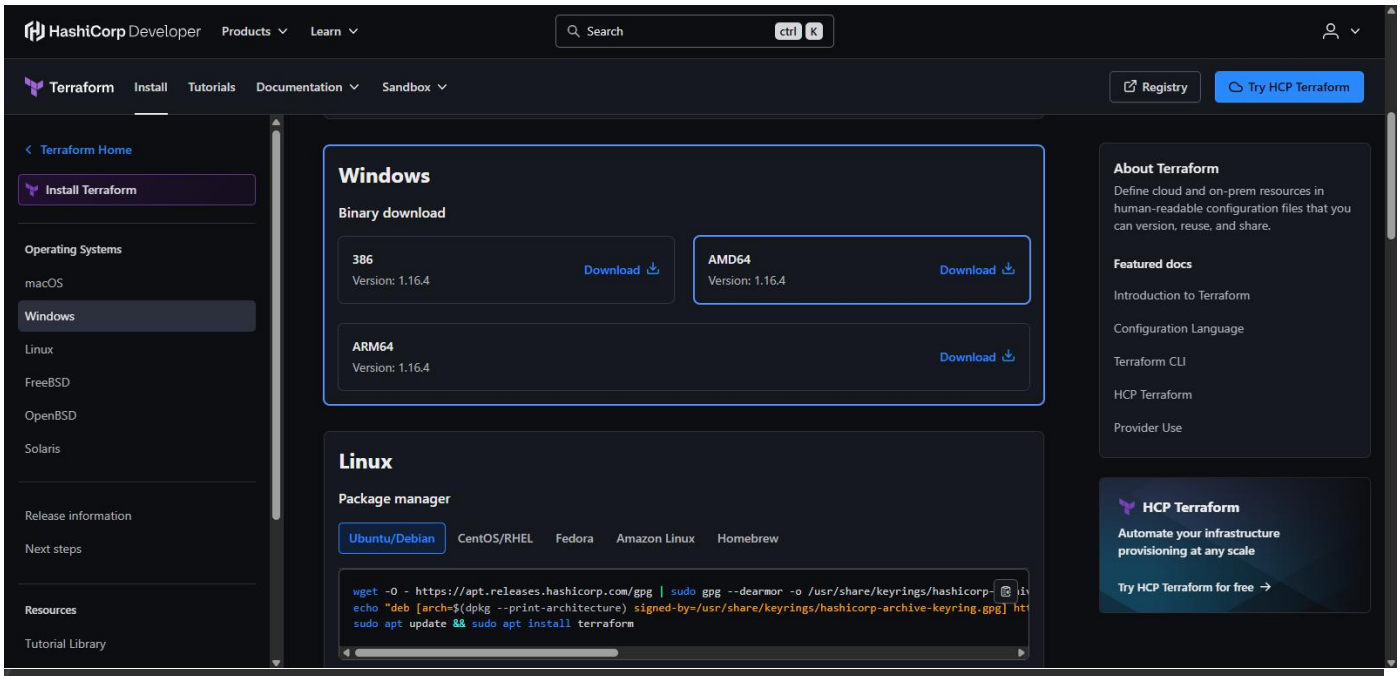
Theory:

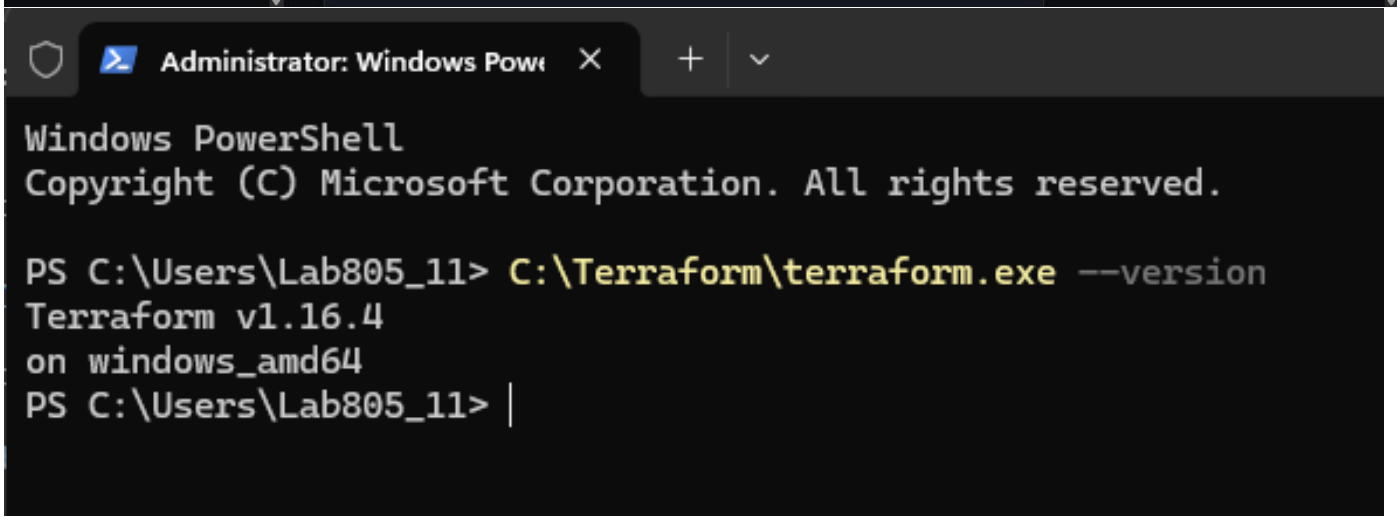
Step 1: Create or verify your AWS account

1. Open the AWS Management Console .
2. Sign in to your AWS account.
3. Select the Mumbai region (ap-south-1).
4. Open the EC2 Dashboard and check that your account is ready to launch instances.

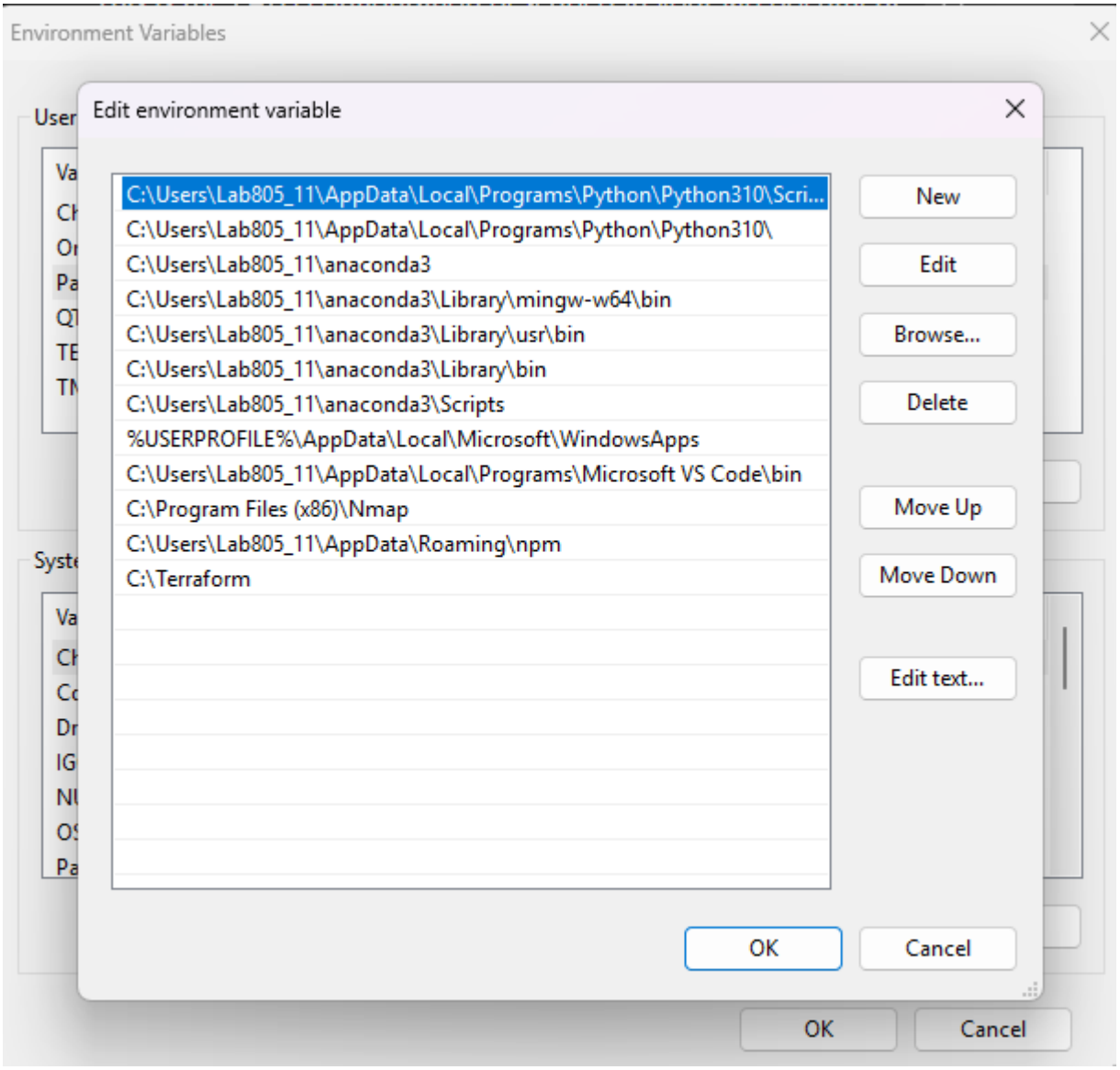
The account must have permissions to create EC2 instances and use the required networking resources. Step 2: Install Terraform on Windows

1. Visit the official Terraform installation page .
2. Download the Windows AMD64 ZIP package.
3. Extract terraform.exe into a folder, for example: C:\Terraform
4. Add C:\Terraform to the Windows PATH environment variable.
5. Open a new Command Prompt or PowerShell window



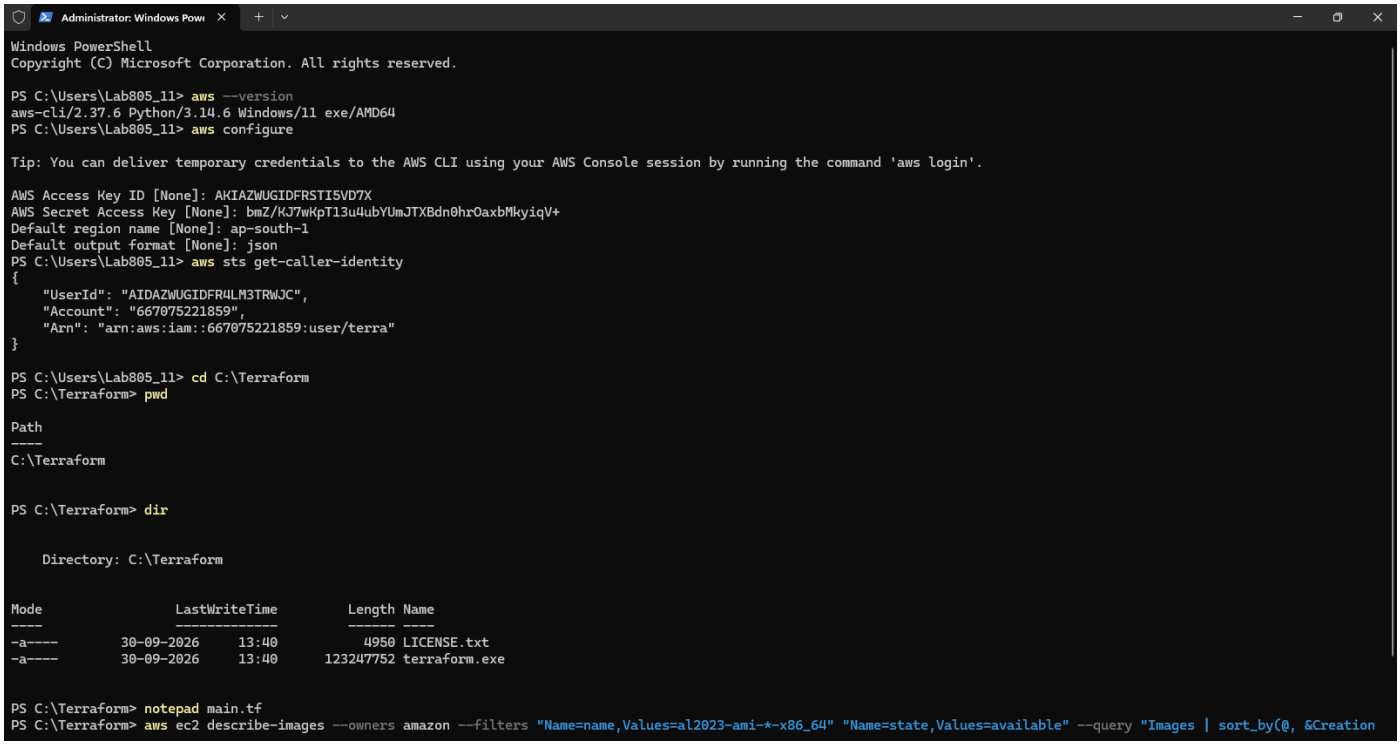


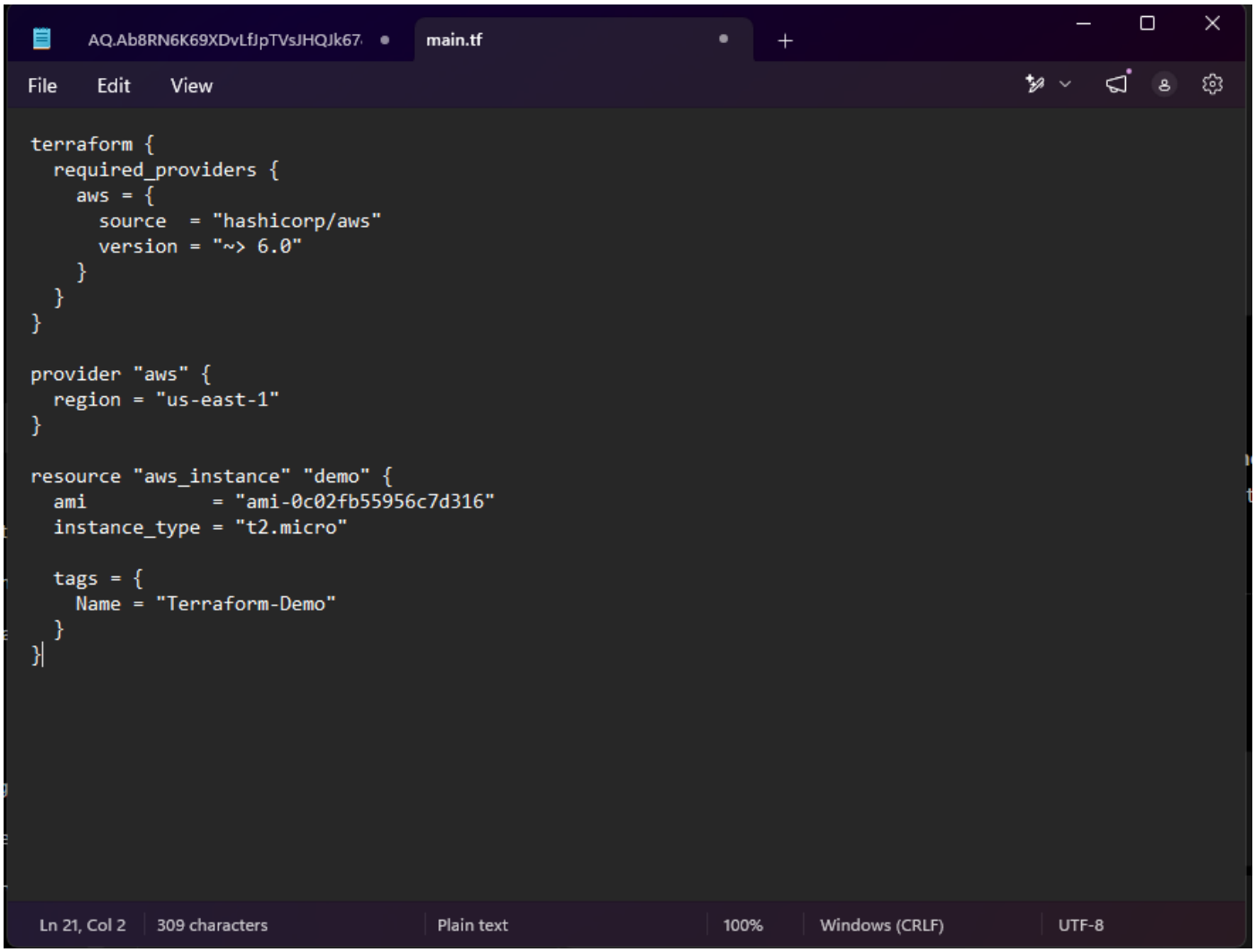
Name : Vansh Sharma Roll no : 2401109 Subject : ASDD Class : T22



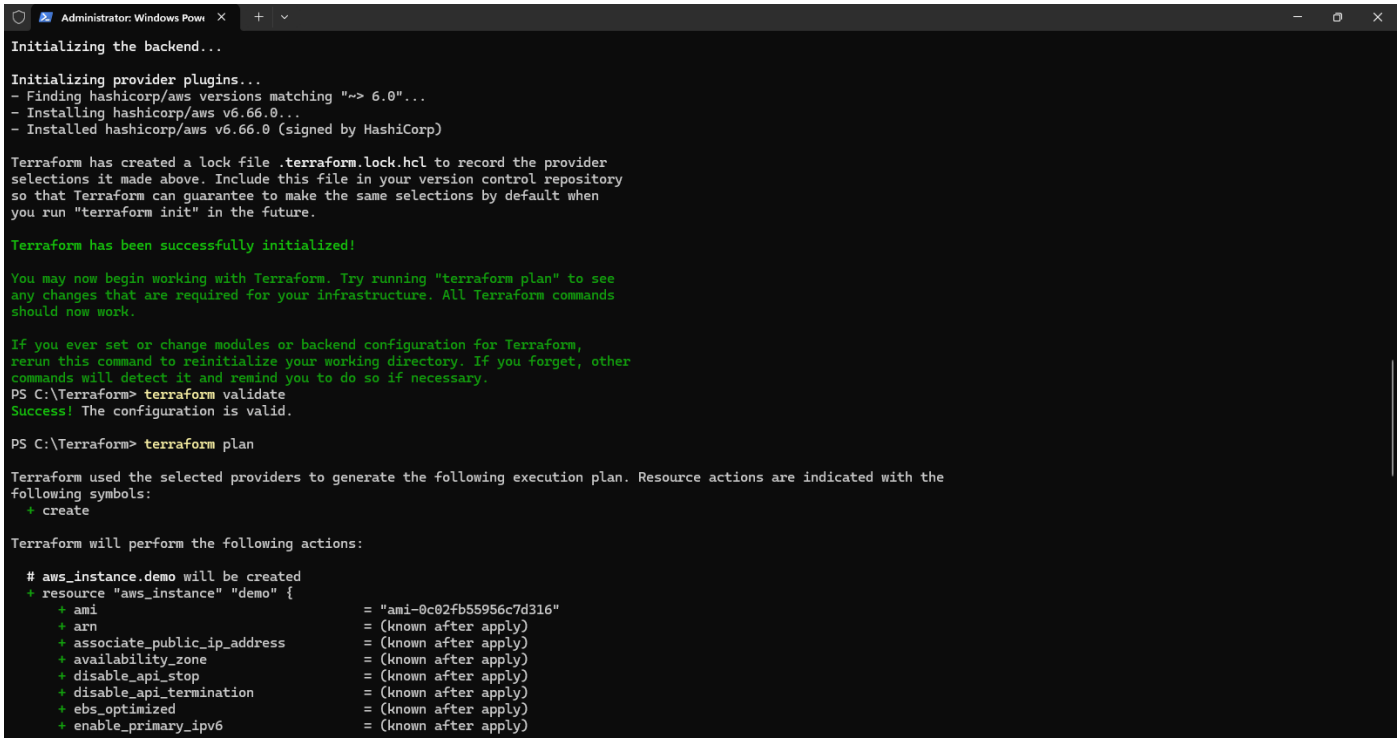


Name : Vansh Sharma Roll no : 2401109 Subject : ASDD Class : T22





Name : Vansh Sharma Roll no : 2401109 Subject : ASDD Class : T22



```
Administrator: Windows Powe

Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 6.0"...
- Installing hashicorp/aws v6.66.0...
- Installed hashicorp/aws v6.66.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

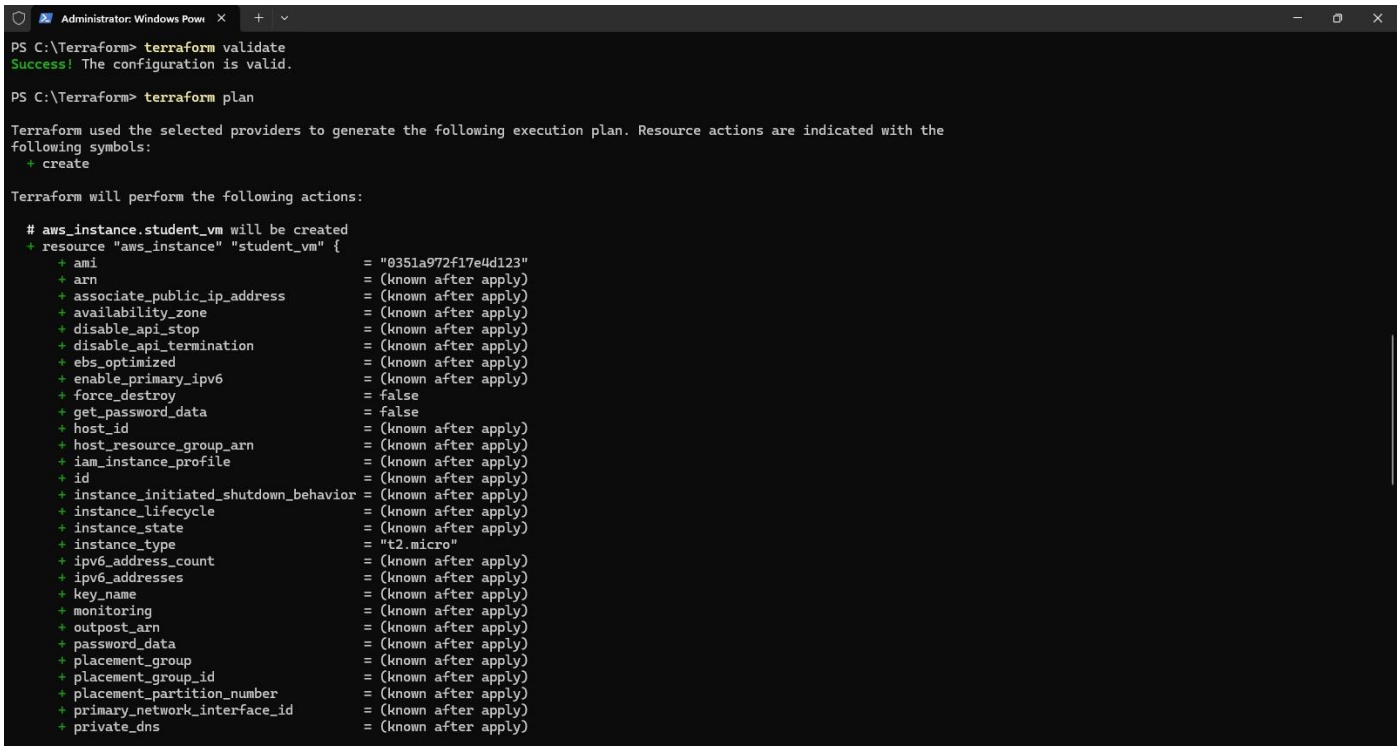
If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\Terraform> terraform validate
Success! The configuration is valid.

PS C:\Terraform> terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
  + create

Terraform will perform the following actions:

  # aws_instance.demo will be created
  + resource "aws_instance" "demo" {
      + ami                                  = "ami-0c02fb55956c7d316"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + enable_primary_ipv6                  = (known after apply)
```



```
Administrator: Windows Powe

PS C:\Terraform> terraform validate
Success! The configuration is valid.

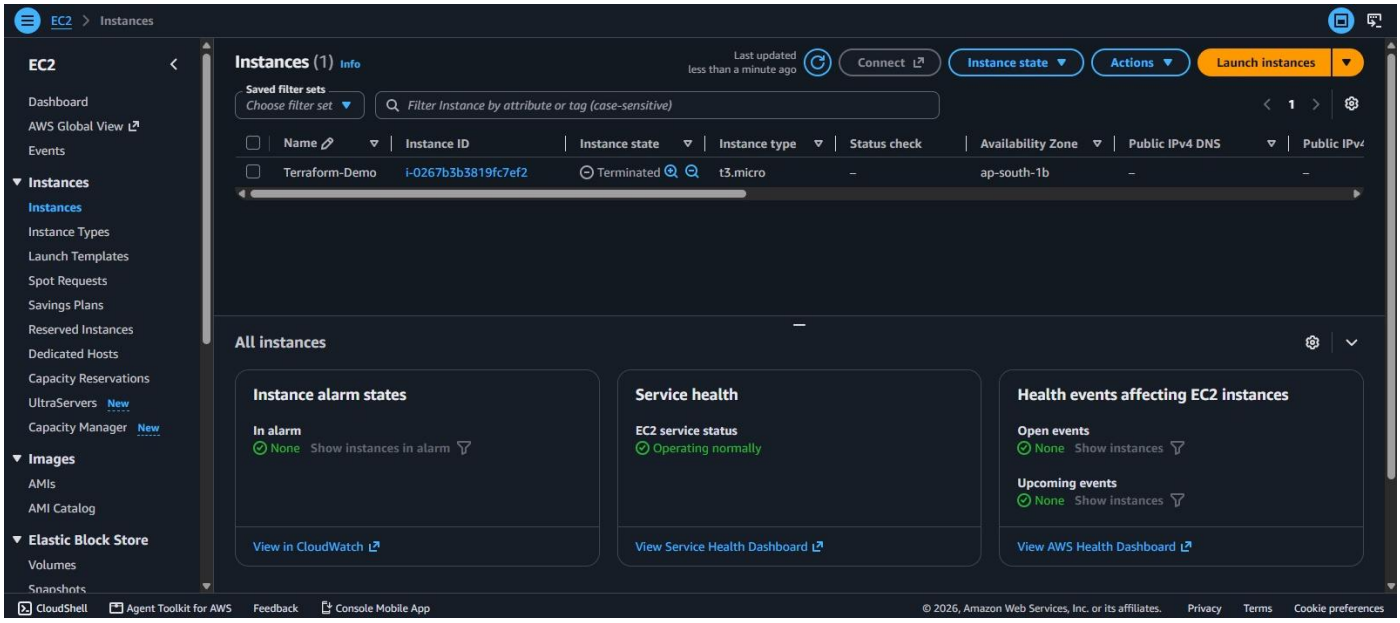
PS C:\Terraform> terraform plan

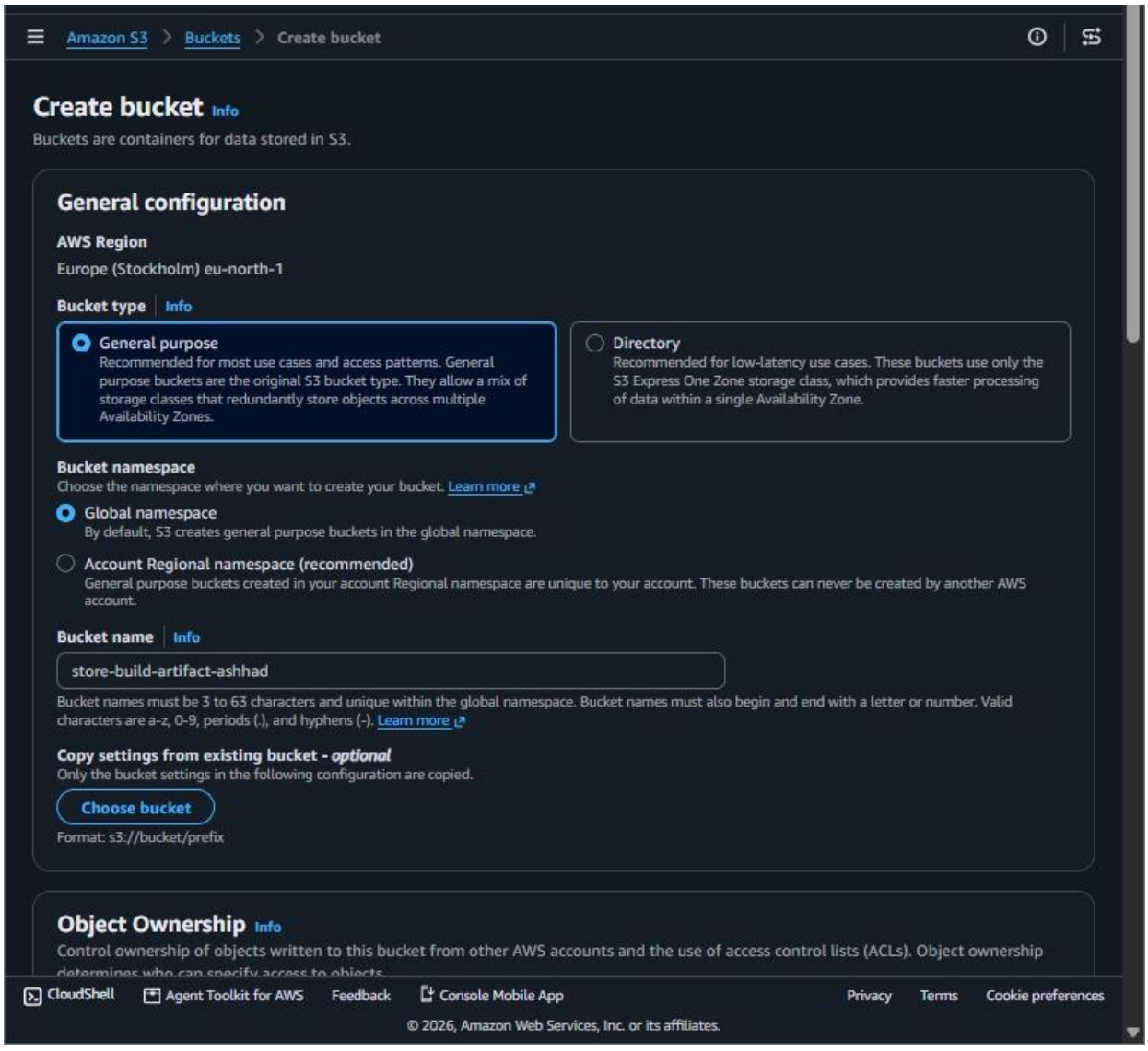
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
  + create

Terraform will perform the following actions:

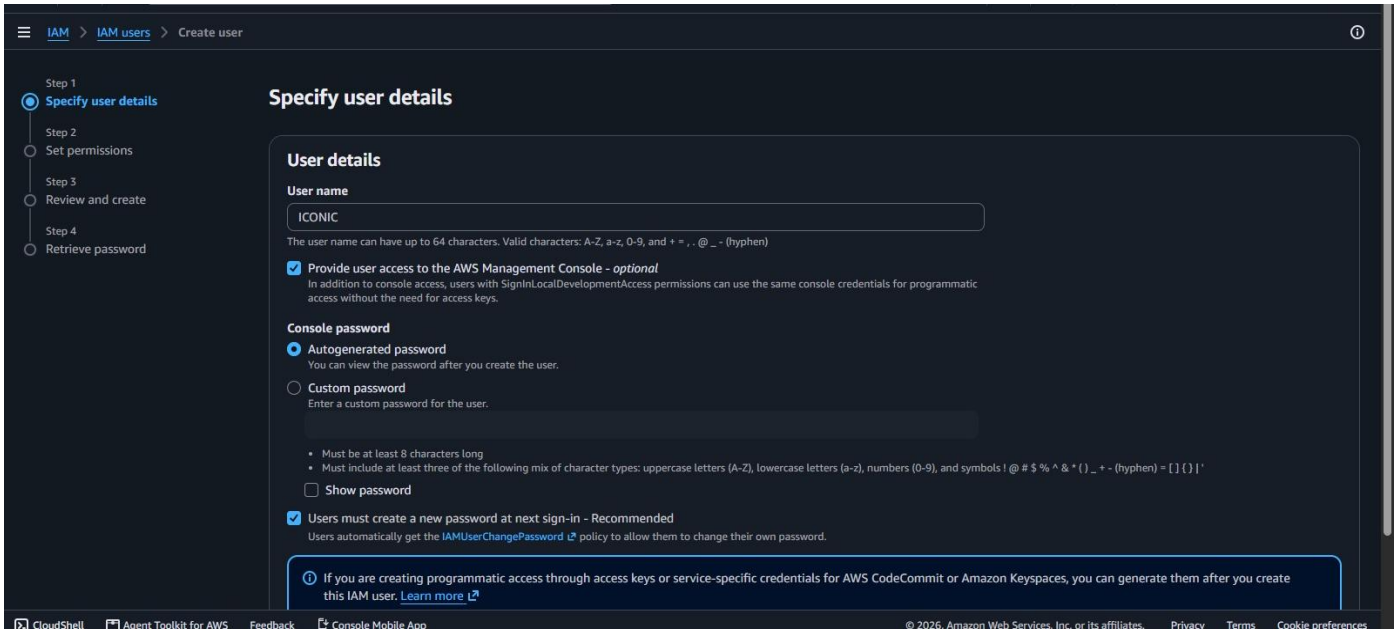
  # aws_instance.student_vm will be created
  + resource "aws_instance" "student_vm" {
      + ami                                  = "0351a972f17e4d123"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + enable_primary_ipv6                  = (known after apply)
      + force_destroy                        = false
      + get_password_data                    = false
      + host_id                              = (known after apply)
      + host_resource_group_arn              = (known after apply)
      + iam_instance_profile                 = (known after apply)
      + id                                   = (known after apply)
      + instance_initiated_shutdown_behavior = (known after apply)
      + instance_lifecycle                   = (known after apply)
      + instance_state                       = (known after apply)
      + instance_type                        = "t2.micro"
      + ipv6_address_count                   = (known after apply)
      + ipv6_addresses                       = (known after apply)
      + key_name                             = (known after apply)
      + monitoring                           = (known after apply)
      + outpost_arn                          = (known after apply)
      + password_data                        = (known after apply)
      + placement_group                      = (known after apply)
      + placement_group_id                   = (known after apply)
      + placement_partition_number           = (known after apply)
      + primary_network_interface_id         = (known after apply)
      + private_dns                          = (known after apply)
```

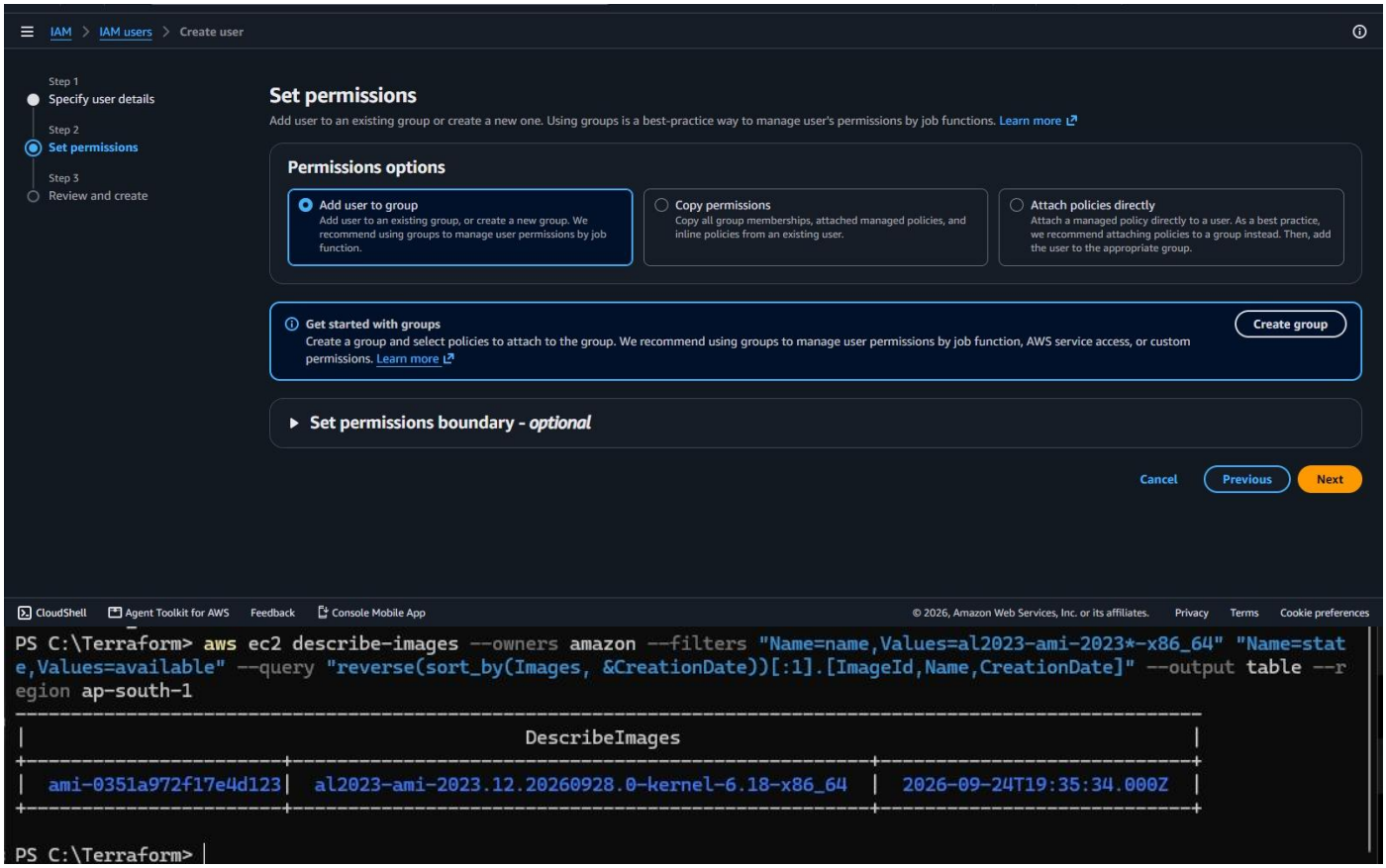
Name : Vansh Sharma Roll no : 2401109 Subject : ASDD Class : T22





Name : Vansh Sharma Roll no : 2401109 Subject : ASDD Class : T22





Name : Vansh Sharma Roll no : 2401109 Subject : ASDD Class : T22

```
PS C:\Terraform> terraform apply
aws_instance.student_vm: Refreshing state... [id=i-0267b3b3819fc7ef2]

Changes to Outputs:
  + instance_id   = "i-0267b3b3819fc7ef2"
  + instance_type = "t3.micro"
  + public_ip     = "13.234.66.58"

You can apply this plan to save these new output values to the Terraform state, without changing any real infrastructure.

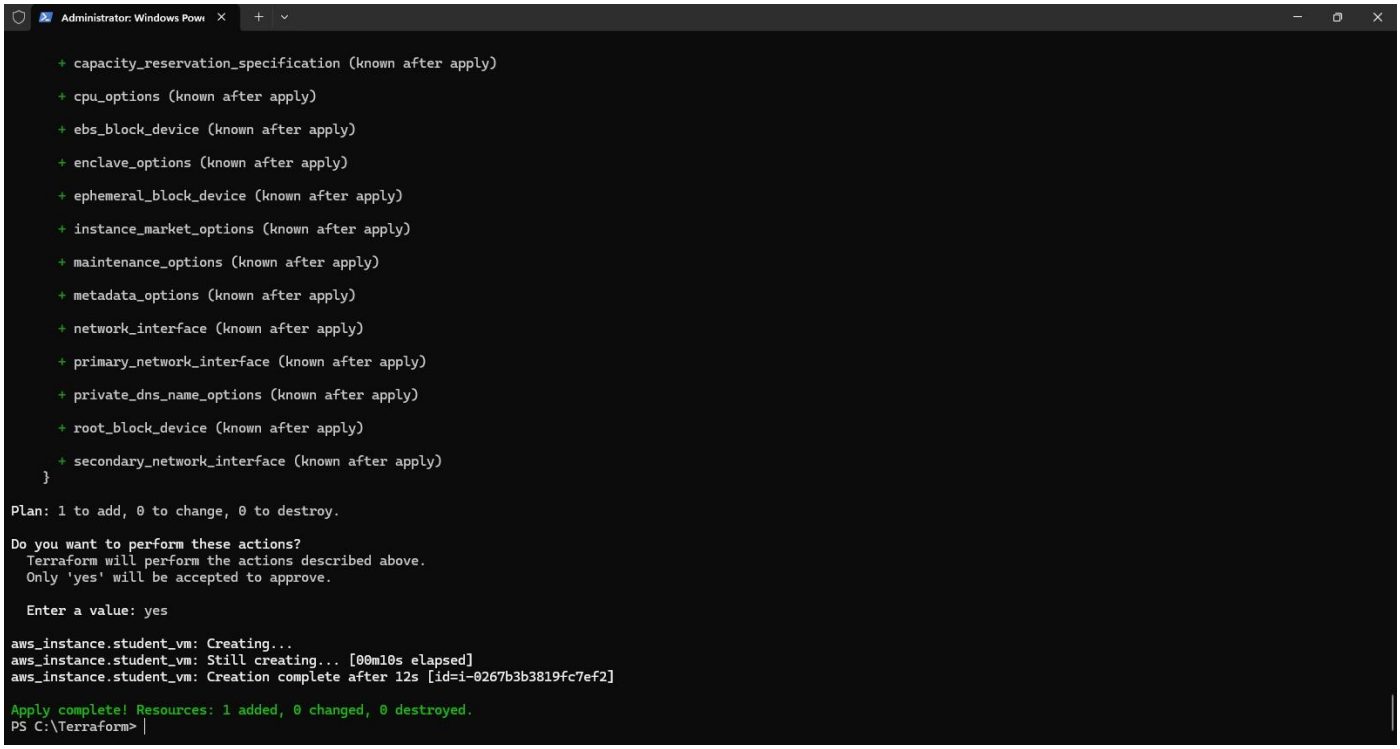
Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes


Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

instance_id = "i-0267b3b3819fc7ef2"
instance_type = "t3.micro"
public_ip = "13.234.66.58"
PS C:\Terraform> terraform output
instance_id = "i-0267b3b3819fc7ef2"
instance_type = "t3.micro"
public_ip = "13.234.66.58"
```



Name : Vansh Sharma Roll no : 2401109 Subject : ASDD Class : T22

```
      - private_dns_name_options {
          - enable_resource_name_dns_a_record    = false -> null
          - enable_resource_name_dns_aaaa_record = false -> null
          - hostname_type                        = "ip-name" -> null
        }

      - root_block_device {
          - delete_on_termination = true -> null
          - device_name           = "/dev/xvda" -> null
          - encrypted             = false -> null
          - iops                  = 3000 -> null
          - tags                  = {} -> null
          - tags_all              = {} -> null
          - throughput            = 125 -> null
          - volume_id             = "vol-0e5e5543f10c3dca9" -> null
          - volume_size           = 8 -> null
          - volume_type           = "gp3" -> null
          # (1 unchanged attribute hidden)
        }
    }

Plan: 0 to add, 0 to change, 1 to destroy.

Changes to Outputs:
  - instance_id   = "i-0267b3b3819fc7ef2" -> null
  - instance_type = "t3.micro" -> null
  - public_ip     = "13.234.66.58" -> null

Do you really want to destroy all resources?
  Terraform will destroy all your managed infrastructure, as shown above.
  There is no undo. Only 'yes' will be accepted to confirm.

  Enter a value: yes

aws_instance.student_vm: Destroying... [id=i-0267b3b3819fc7ef2]
aws_instance.student_vm: Still destroying... [id=i-0267b3b3819fc7ef2, 00m10s elapsed]
aws_instance.student_vm: Still destroying... [id=i-0267b3b3819fc7ef2, 00m20s elapsed]
aws_instance.student_vm: Destruction complete after 30s

Destroy complete! Resources: 1 destroyed.
PS C:\Terraform>
```

.tf files in the same folder are treated as **one Terraform configuration**. The filename does not determine what gets executed.

Terraform destroy can be used to withdraw access from the infrastructure. Student must do this.

Output: **Clear and visible screenshots must be attached.**

12. Learning Outcomes Achieved :

Students gained practical knowledge of using Terraform to define, provision, and manage AWS cloud infrastructure through Infrastructure as Code (IaC).

13. Conclusion:

1. **Applications of the studied technique in industry**
   Terraform enables automated and consistent provisioning of cloud infrastructure, supporting efficient deployment and management of AWS resources.

14. Engineering Relevance

The experiment demonstrates Infrastructure as Code practices that improve scalability, reliability, repeatability, and efficiency in cloud-based engineering.

15. Skills Developed

Students developed skills in writing Terraform configurations, executing Terraform commands, and provisioning AWS resources using Terraform.